CHAPTER 1

Recurrent neural networks and sequence modeling

In Chapter **??**, we developed the continuous mathematical foundation of neural representations, proving how non-linear activations prevent matrix collapse and how reverse-mode automatic differentiation computes parameter sensitivities across complex computational graphs. Yet, our architectural analysis revealed an irreducible structural impasse: feedforward networks are intrinsically static. By operating exclusively over fixed-dimensional Euclidean vectors $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, feedforward networks treat each input in isolation, possess no persistent internal memory, and enforce rigid, position-specific parameterizations that fail to generalize across sequence positions.

Human natural language and software source code, however, are inherently sequential and elastic. Words derive their meaning from preceding and succeeding context, syntactic rules govern grammatical agreement across distant clauses, and execution traces unfold chronologically over time. To process sequential phenomena effectively, deep learning models must encode the underlying physical symmetries of time-series data: *temporal stationarity* and *causal recurrence*.

This chapter develops the theoretical and practical foundations of recurrent sequence modeling. We begin by examining the failure modes of feedforward networks and formalizing the inductive biases of convolutional and recurrent architectures through the twin symmetries of translation equivariance and temporal parameter sharing. We then derive the mechanics of the simple recurrent cell, prove the mathematical pathology of vanishing and exploding gradients in Backpropagation Through Time (BPTT) via Pascanu's spectral radius theorem, analyze character-level training dynamics on a classic literary corpus, unpack the Constant Error Carousel (CEC) of gated LSTM and GRU architectures, and evaluate Sequence-to-Sequence pipelines with Bahdanau additive attention.

1.1 Sequential data and constraints of feed-forward neural networks

In Chapter **??**, we formulated the mathematical and architectural foundations of deep feedforward neural networks (FCNNs) and backpropagation across computational DAGs. As established at the conclusion of that chapter, however, feedforward architectures operate under a rigid, memoryless paradigm: they assume fixed-dimensional Euclidean inputs $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, possess no internal recurrence or persistent temporal state across sequential time steps, and cannot enforce grammatical agreement across temporal horizons.

In standard deep learning, a Fully-Connected Neural Network (FCNN) or Multi-Layer Perceptron is represented as a cascade of weight matrices interleaved with non-linear activation functions:

$$f(\mathbf{x}) = \sigma\left(\mathbf{W}_{\text{out}}\,\text{ReLU}\left(\mathbf{W}_2\,\text{ReLU}\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2\right) + \mathbf{b}_{\text{out}}\right). \tag{1.1}$$

While FCNNs excel on tabular datasets with fixed, predefined attributes, they exhibit three critical architectural constraints when applied to natural language and temporal sequences:

1. **Rigid Input Dimensionality**: The input dimensionality is rigidly fixed by the column dimension of the input weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_{\text{in}}}$. While structured tabular data naturally adheres to fixed attribute vectors, natural language utterances exhibit variable sequence lengths T (spanning from single-word imperatives to multi-clause legal sentences). Flattening variable-length sequences into a static vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ requires choosing an arbitrary upper bound $T_{\max}$, truncating longer sequences and zero-padding shorter ones, which wastes memory and degrades efficiency.

2. **Absence of Parameter Sharing and Translation Invariance**: In an FCNN, each coordinate in the flattened input vector is governed by independent weight parameters. Consequently, identical syntactic or semantic constructs appearing at different positions in a sentence (such as a noun phrase located at token position 1 versus token position 10) must be learned independently from scratch, destroying sample efficiency.

3. **Rigid Output Dimensionality**: The output dimensionality is strictly predetermined by the row dimension of the final projection matrix $\mathbf{W}_{\text{out}}$. In generative sequential tasks such as machine translation, summarization, and dialogue generation, the target output length is unknown a priori and varies dynamically relative to the input (for example, translating a 5-word English sentence into a 4-word Italian clause). Feedforward networks cannot dynamically modulate their output dimension to emit variable-length token sequences.

1.1.1 Inductive biases and the twin physical symmetries: CNNs vs. RNNs

As articulated in modern deep learning theory [**? ?**], unconstrained multi-layer perceptrons possess minimal *inductive bias*: they treat every input dimension as an arbitrary coordinate, ignoring any topological or temporal relationships between adjacent dimensions. To scale deep learning to complex domains, architectures must incorporate inductive biases that exploit the intrinsic **physical symmetries** of the underlying data:

Spatial translation equivariance in CNNs

In spatial domains (such as 2D images), the data-generating distribution is characterized by two fundamental symmetries:

- **Locality**: Pixels that are spatially adjacent exhibit strong mutual statistical correlations, whereas pixels separated by large Euclidean distances are approximately independent.

- **Translation Invariance / Equivariance**: Shifting an input feature map $\mathbf{x}$ by spatial displacement vector $\mathbf{g}$ should produce a correspondingly shifted feature representation $f(g(\mathbf{x})) = g(f(\mathbf{x}))$.

Imposing locality and translation equivariance forces the dense weight matrix $\mathbf{W} \in \mathbb{R}^{D_{\text{out}} \times D_{\text{in}}}$ to collapse into a localized discrete convolution operator $\mathbf{K} \in \mathbb{R}^{k \times k}$ ($k \ll \sqrt{D_{\text{in}}}$). By sliding this compact kernel across all spatial coordinates, the parameter footprint is reduced by several orders of magnitude, enabling sample-efficient visual recognition.

Temporal stationarity and parameter sharing in RNNs

In temporal and linguistic domains, the governing symmetry is **temporal stationarity**:

$$\mathbb{P}(X_t \mid X_{t-1}, \dots, X_1) \approx \mathcal{T}(X_t, X_{t-1}, \dots; \Theta), \tag{1.2}$$

meaning that the fundamental grammatical rules, syntactic patterns, and conditional transition probabilities of a language do not alter their mathematical structure simply because an utterance takes place at token index $t = 1$ versus index $t = 50$.

While an FCNN requires separate, independent weight matrices $\mathbf{W}_t \in \mathbb{R}^{d \times d}$ for each position $t \in \{1, \dots, T\}$—causing parameter count to scale as $O(T \cdot d^2)$—a recurrent architecture models sequence progression through a stationary state-transition mapping $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \Theta)$. By sharing the parameter set $\Theta = \{\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h\}$ across all time steps, the parameter footprint remains completely invariant to sequence length: $O(d_h^2 + d_h d_{\text{in}})$.

1D Temporal Convolutions vs. Recurrence

Simon Prince [**?**] highlights an alternative formulation: 1D Temporal Convolutions (e.g., WaveNet, Temporal Convolutional Networks / TCNs). Instead of maintaining an evolving recurrent state $\mathbf{h}_t$, a 1D TCN applies causal dilated 1D convolutions over time. While 1D CNNs achieve $O(1)$ sequential parallelization during training, their effective memory horizon is strictly bounded by their receptive field, scaling exponentially with network depth as $O(K^L)$. In contrast, recurrent networks maintain a theoretically unbounded memory horizon within a single compact state $\mathbf{h}_t$, albeit at the expense of strictly sequential training.

Table **??** formalizes the architectural trade-offs across the four foundational deep learning paradigms.

Table 1.1: Architectural comparison across deep learning paradigms: Inductive biases, computational complexity, and sequence handling.

Dimension	Feedforward (FCNN)	1D Temporal CNN (TCN)	Recurrent (RNN/LSTM)	Transformer
Input Elasticity	Rigid ($T_{\max}$ fixed)	Elastic (up to receptive field)	Elastic (unbounded variable length)	Elastic (bounded by context window $L_{\max}$)
Parameter Sharing	None (position-specific)	1D sliding causal kernel	Replay over time (temporal recurrence)	Shared projection weights across tokens
Training Latency	$O(1)$ parallel matrix pass	$O(1)$ parallel causal convolutions	$O(T)$ strictly sequential	$O(1)$ parallel self-attention
Receptive Field	Memoryless (fixed window)	Bounded by depth $O(K^L)$	Theoretically unbounded ($\mathbf{h}_t$)	Direct $O(1)$ all-to-all token routing
Inference Cost / Step	Re-evaluates entire window	Causal FIFO buffer rolling	$O(1)$ update step $f(\mathbf{h}_{t-1}, \mathbf{x}_t)$	KV Cache lookup $O(T)$ operations

1.2 The recurrent neural network cell and weight sharing

To overcome the constraints of FCNNs, **Recurrent Neural Networks (RNNs)**—first formalized as the Simple Recurrent Network in [**?**]—process sequential data element by element, maintaining an internal continuous *hidden state* $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as a recurrent memory summarizing the sequence history observed up to time step t.

1.2.1 Recurrent cell architecture

At each discrete time step $t \in \{1, 2, \ldots, T\}$, the recurrent cell is characterized by two inputs and two outputs:

- **Input 1**: The current sequence element $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ (e.g., a one-hot vector or continuous embedding).
- **Input 2**: The hidden state produced in the previous time step $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$.
- **Output 1**: The model's prediction for the current time step $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_{\text{out}}}$.
- **Output 2**: The updated hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$, which is recursively forwarded to time step $t+1$.

1.2.2 Mathematical formulation with explicit tensor dimensions

Internally, the vanilla RNN cell comprises an input projection layer, a recurrent state transition layer, and an output projection layer:

$$\underbrace{\mathbf{h}_t}_{[d_h \times 1]} = \tanh\left(\underbrace{\mathbf{W}_{hh}}_{[d_h \times d_h]}\underbrace{\mathbf{h}_{t-1}}_{[d_h \times 1]} + \underbrace{\mathbf{W}_{xh}}_{[d_h \times d_{\text{in}}]}\underbrace{\mathbf{x}_t}_{[d_{\text{in}} \times 1]} + \underbrace{\mathbf{b}_h}_{[d_h \times 1]}\right), \tag{1.3}$$

$$\underbrace{\hat{\mathbf{y}}_t}_{[V \times 1]} = \text{softmax}\left(\underbrace{\mathbf{W}_{hy}}_{[V \times d_h]}\underbrace{\mathbf{h}_t}_{[d_h \times 1]} + \underbrace{\mathbf{b}_y}_{[V \times 1]}\right). \tag{1.4}$$

1.2.3 Weight sharing across time

The defining feature of RNNs is **temporal weight sharing**. The parameter matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are *strictly identical* across all time steps $t \in \{1, \ldots, T\}$. Because the same weights are applied regardless of time step, the model develops **translation invariance**: if the same context occurs at step $t = 2$ or step $t = 50$, the recurrent cell behaves identically. The network learns grammatical rules (such as capitalizing the first letter after a quotation mark) once, and applies that rule everywhere in the text.

*As visualized in geometric visual analyses [**?**], the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ can be understood geometrically as the position of a particle traveling through a continuous d_h-dimensional phase space:*

- At rest ($t = 0$), the system sits at initial coordinate $\mathbf{h}_{-1} = \mathbf{0}$.
- When each discrete input token $\mathbf{x}_t$ arrives, the linear projection $\mathbf{W}_{xh}\mathbf{x}_t$ applies an external directional impulse or "vector nudge" to the particle.
- Simultaneously, the recurrent transformation $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ rotates, scales, and folds the previous position, while the non-linear squashing activation tanh confines the entire state space within the hypercube $(-1, 1)^{d_h}$.

- The unfolding sequence of text traces a smooth continuous trajectory through state space. Regions of this phase space function as **attractor basins**: for instance, reading an opening quote mark (") knocks the state into a specific sub-manifold where the emission projection $\mathbf{W}_{hy}$ naturally maps coordinates into uppercase character distributions. The state remains trapped in this basin until an exit trigger (such as a whitespace or subsequent character) forces the trajectory out into the lowercase manifold.

1.2.4 Unrolling the recurrent cell across time

To trace how an RNN processes a sequence, we *unroll* the recursive feedback loop across discrete time steps $t = 0, 1, \ldots, T$.

Consider the character casing task:

- Input sequence: `"you are very kind, mr.  holmes, but i cannot do that"`
- Target sequence: `"You are very kind, Mr.  Holmes, but I cannot do that"`

Each ASCII character is represented as a 256-dimensional one-hot vector $\mathbf{x}_t \in \{0, 1\}^{256}$:

- **Step** $t = 0$: Receives input $s_0 =$ `"` with initial state $\mathbf{h}_{-1} = \mathbf{0}$. The cell produces state $\mathbf{h}_0$. The network realizes: *"We just encountered an opening quote (`"`), which signals the beginning of dialogue; the following character should be capitalized."*
- **Step** $t = 1$: Receives input $s_1 =$ `y` and prior state $\mathbf{h}_0$. The network emits output $\hat{y}_1 =$ `Y` and produces updated state $\mathbf{h}_1$. The state records: *"We just emitted a capitalized letter at the start of dialogue; following letters in this word should probably be lowercase."*
- **Step** $t = 2$: Receives input $s_2 =$ `o` and prior state $\mathbf{h}_1$. The network emits output $\hat{y}_2 =$ `o` and produces state $\mathbf{h}_2$, continuing lowercase word progression.

Figure **??** illustrates this unrolled computational structure and the backward flow of gradients.

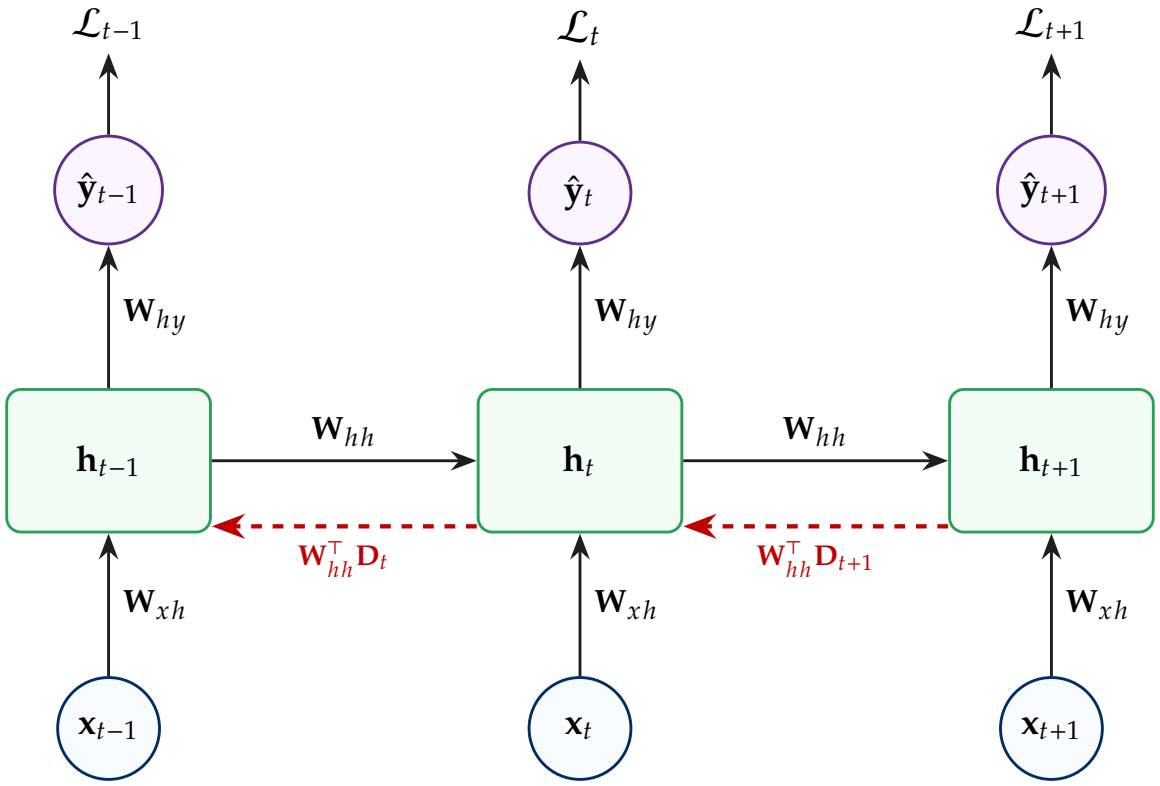


Figure 1.1: Unrolling an RNN cell across time steps $t-1, t, t+1$. Solid black arrows show the forward propagation of inputs $\mathbf{x}_t$ and recurrent states $\mathbf{h}_t$. Dashed red arrows trace Backpropagation Through Time (BPTT), where gradients accumulate across time via repeated multiplications by the Jacobian $\mathbf{W}_{hh}^\top \mathbf{D}_t$.

1.3 Backpropagation through time and the proof of vanishing gradients

When an RNN processes a sequence of length T, the computational graph is unrolled T times. The total sequence loss is the sum of cross-entropy losses incurred at each time step:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t. \tag{1.5}$$

To optimize the shared recurrent weight matrix $\mathbf{W}_{hh}$ via Gradient Descent, we apply **Backpropagation Through Time (BPTT)**, first formalized for recurrent structures in [**?**]. The fundamental difficulty of learning long-term dependencies through gradient descent was rigorously analyzed in [**?**] and [**?**]. The gradient of the total loss with respect to $\mathbf{W}_{hh}$ accumulates the contributions across all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \tag{1.6}$$

where $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denotes the direct derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$ treating $\mathbf{h}_{k-1}$ as constant.

The temporal transmission term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the sensitivity of state $\mathbf{h}_t$ to an earlier state $\mathbf{h}_k$ ($t > k$). By the multivariate chain rule, this sensitivity decomposes into a product of single-step Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \tag{1.7}$$

1.3.1 Rigorous mathematical proof of gradient pathology

We now derive the explicit single-step Jacobian and prove why vanilla RNNs suffer from vanishing and exploding gradients.

Theorem 1.1: *Jacobian product and spectral radius gradient decay*

Let the recurrent state transition be $\mathbf{h}_j = \tanh(\mathbf{z}_j)$ where $\mathbf{z}_j = \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h$ with $\mathbf{h}_j \in \mathbb{R}^{d_h}$ and $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$. Under standard column-vector convention, the single-step forward temporal Jacobian $\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \in \mathbb{R}^{d_h \times d_h}$ is:

$$\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh} = \mathbf{D}_j \mathbf{W}_{hh}, \tag{1.8}$$

where $\mathbf{D}_j = \operatorname{diag}(1 - \tanh^2(\mathbf{z}_j)) \in \mathbb{R}^{d_h \times d_h}$ is a diagonal matrix containing the local derivatives of the hyperbolic tangent function. Consequently, the multi-step temporal transmission evaluates to:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}. \tag{1.9}$$

When transmitting the gradient of a scalar loss $\mathcal{L}_t$ backward through time, the column-vector gradient $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t \in \mathbb{R}^{d_h}$ satisfies the adjoint recurrence $\nabla_{\mathbf{h}_{j-1}} \mathcal{L}_t = \mathbf{W}_{hh}^\top \mathbf{D}_j \nabla_{\mathbf{h}_j} \mathcal{L}_t$. Let $\lambda_{\max} = \|\mathbf{W}_{hh}\|_2 = \sigma_{\max}(\mathbf{W}_{hh})$ denote the spectral norm (maximum singular value) of $\mathbf{W}_{hh}$, and let $\rho(\mathbf{W}_{hh}) = \max_i |\lambda_i|$ denote its spectral radius. The Euclidean norm of the transmitted gradient is bounded by:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| (\lambda_{\max})^{t-k}. \tag{1.10}$$

Proof. Differentiating the i-th component of $\mathbf{h}_j$ with respect to the m-th component of $\mathbf{h}_{j-1}$:

$$\frac{\partial h_{j,i}}{\partial h_{j-1,m}} = \frac{\partial \tanh(z_{j,i})}{\partial z_{j,i}} \frac{\partial z_{j,i}}{\partial h_{j-1,m}} = (1 - h_{j,i}^2) W_{hh,i,m}.$$

In matrix notation, the entry in row i and column m of $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ is $(1 - h_{j,i}^2) W_{hh,i,m} = (\mathbf{D}_j)_{ii} W_{hh,i,m}$, which corresponds exactly to the matrix product $\mathbf{D}_j \mathbf{W}_{hh}$. Taking the transpose for backward column-vector gradient propagation yields $(\mathbf{D}_j \mathbf{W}_{hh})^\top = \mathbf{W}_{hh}^\top \mathbf{D}_j$, matching the backward adjoint arrows in Figure **??**. Because $|\tanh'(z)| = 1 - \tanh^2(z) \in [0, 1]$, the operator 2-norm of each diagonal matrix satisfies $\|\mathbf{D}_j\|_2 \leq 1$. Applying the sub-multiplicative property of matrix norms to Equation (**??**):

$$\begin{aligned} \|\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}\|_2 = \|\prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}\|_2 &\leq \prod_{j=k+1}^{t} \|\mathbf{D}_j\|_2 \|\mathbf{W}_{hh}\|_2 \\ &\leq \prod_{j=k+1}^{t} 1 \cdot \lambda_{\max} = (\lambda_{\max})^{t-k}. \end{aligned}$$

Multiplying by $\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\|_2$ yields Equation (**??**). □

1.3.2 Two pathological regimes: vanishing and exploding gradients

As proved by Pascanu et al. [**?**], the asymptotic behavior of the temporal Jacobian product $\prod_{j=k+1}^{t} \mathbf{D}_j \mathbf{W}_{hh}$ bifurcates into two destructive regimes dictated by the spectrum of $\mathbf{W}_{hh}$:

1. **Vanishing Gradient ($\lambda_{\max} < 1$ or $\rho(\mathbf{W}_{hh}) < 1$)**: If the largest singular value or spectral radius of $\mathbf{W}_{hh}$ is strictly less than 1 (e.g., $\lambda_{\max} = 0.9$), over a temporal distance of 50 steps:

   $$0.9^{50} \approx 0.00515 \quad \text{(over 99.5\% of the gradient signal is extinguished).}$$

   Over 100 steps, $0.9^{100} \approx 0.000026$. By Gelfand's formula ($\lim_{n\to\infty} \|\mathbf{W}^n\|^{1/n} = \rho(\mathbf{W})$), the gradient decays exponentially to zero. Parameter updates from distal tokens vanish completely, meaning the network cannot learn long-range temporal dependencies. Saturated activations ($\tanh'(z) \to 0$) accelerate this decay exponentially.

2. **Exploding Gradient ($\lambda_{\max} > 1$ or $\rho(\mathbf{W}_{hh}) > 1$)**: If the singular values exceed 1 (e.g., $\lambda_{\max} = 1.1$), over 50 steps:

   $$1.1^{50} \approx 117.39.$$

   Over 100 steps, $1.1^{100} \approx 13{,}780$. Gradients grow exponentially along the dominant eigenvectors, destabilizing training, inducing severe numerical overflow, and producing `NaN` parameter corruptions.

Global norm gradient clipping

To combat exploding gradients, Pascanu et al. [**?**] introduced the technique of *global norm clipping*. Instead of clipping individual scalar gradient elements independently (which alters the geometric direction of the descent vector), global norm clipping rescales the collective parameter gradient vector $\mathbf{g} = \nabla_\Theta \mathcal{L}$ whenever its total Euclidean norm exceeds a predefined threshold $\gamma > 0$:

$$\mathbf{g} \leftarrow \min\left(1, \frac{\gamma}{\|\mathbf{g}\|_2}\right) \mathbf{g}. \tag{1.11}$$

Geometrically, Equation (**??**) leaves the gradient direction strictly unaltered ($\frac{\mathbf{g}}{\|\mathbf{g}\|}$ is preserved) while projecting the step magnitude onto the hypersphere of radius γ. This prevents gradient descent from taking catastrophic, destructive leaps across steep loss ravines and cliff-like bifurcations.

Long-term dependency degradation

Because hidden states are rewritten at every time step without persistent memory protection, vanilla RNNs suffer from catastrophic recency bias: they remember only the most recent tokens. When tested on long-range word prediction:

*"The **mouse** was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the _____"*

the target word is unambiguously *"mouse"*. However, vanilla RNNs fail completely because the hidden state has overwritten the subject *"mouse"* across the intervening 20 tokens.

1.4 Case correction demonstration: Sherlock Holmes training dynamics

To demonstrate how an RNN learns sequential syntax in practice, we examine an empirical case study trained on Arthur Conan Doyle's *The Adventures of Sherlock Holmes*:

Experimental configuration

- **Corpus**: Raw text from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*.
- **Sequence Length**: Subsequences sliced between 10 and 100 ASCII characters.
- **Token Representation**: 256-dimensional one-hot vectors ($d_{\text{in}} = 256$).
- **Architecture**: Single-layer vanilla RNN with hidden dimension $d_h = 32$.
- **Optimization**: Adam optimizer (lr = 0.001), batch size 32, trained for 10 epochs.
- **Padding Mechanism**: Null bytes (`0x00`, rendered visually as `X`) used to pad variable-length sequences to 100 characters.

Evaluation benchmark sentence

The model is evaluated after each epoch on a fixed unseen test string:

- **Input Test Sequence**: `upon his pale face.  "it may be so, or it may not, mr. holmes," said he, "but if you are so very sharp`
- **Target Test Sequence**: `upon his pale face.  "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp`

Qualitative phase transitions across training epochs

Table **??** records the loss trajectory and qualitative phase transitions across training:

1.4.1 Character-level modeling, temperature sampling, and hidden state interpretability

The Sherlock Holmes casing experiment exemplifies a broader paradigm analyzed in the landmark study *"The Unreasonable Effectiveness of Recurrent Neural Networks"* [**?**]: **character-level sequence modeling**.

Unlike word-level models that require complex tokenizers, morphological analyzers, and massive vocabularies ($V \geq 50{,}000$), character-level RNNs operate over an extremely compact alphabet (e.g., standard 256-dimensional ASCII).

- There are **zero Out-of-Vocabulary (OOV) tokens**: any arbitrary string, code snippet, mathematical formula, or neologism can be represented as an sequence of bytes.
- The model is forced to discover the hierarchical structure of language entirely from scratch: it must learn how characters form words, that words are separated by spaces, how punctuation delimits clauses, and how grammatical agreement spans across clauses.

Table 1.2: Epoch-by-epoch loss progression and qualitative phase transitions on the Sherlock Holmes casing task.

Epoch	Loss	Observed Generation & Qualitative Linguistic Transition
01	3.1681	`XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 1 (Null-byte frequency)*: The model discovers that null padding bytes (`0x00` / `X`) appear with high statistical frequency due to padding imbalance, predicting all `X`'s.
02	2.3334	`eXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` *Phase 2 (Vowel emergence)*: The model predicts `e` (the most frequent character in English orthography) alongside padding characters.
03	1.7436	`tst X s te e eee t e e e te se ee se e e ee eee XXXee` *Phase 3 (Whitespace and unigrams)*: The model learns that whitespace characters occur regularly, predicting spaces, `e`, `t`, and `s`.
04	0.9519	`upon his pale face.  cit may be sos or it may nots mr.  holresst` *Phase 4 (The Identity Breakthrough)*: A massive drop in loss occurs when the network learns to forward lowercase letters directly from input to output (`W.grad` captures the identity diagonal).
05–06	0.3579	`upon his pale face.  "it may be so, or it may not, mr.  holmes,"` *Phase 5 (Punctuation and Syntax)*: The model captures quotation marks (`"`), commas, and spaces accurately.
07–08	0.1009	`upon his pale face.  "It may be so, or it may not, mr.  holmes,"` *Phase 6 (Quote Capitalization)*: The model learns that characters immediately following opening quotes (`"`) must be capitalized (`"It`).
09–10	0.0515	`upon his pale face.  "It may be so, or it may not, mr.  Holmes,"` *Phase 7 (Period Capitalization and Causal Blindness)*: The model capitalizes `Holmes` because it follows a period (`.`). Crucially, the model **fails to capitalize** `mr.` because it cannot see into the future! It does not know that `m` begins the title `Mr.` until the `r.` arrives.

During text generation, the RNN emits an unnormalized logit vector $\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^V$. Rather than taking the deterministic arg max, we convert logits into a sampling distribution modulated by a scalar hyperparameter: the **temperature** $T > 0$:

$$\mathbb{P}(y_t = c \mid \mathbf{h}_t) = \frac{\exp\left(\frac{z_{t,c}}{T}\right)}{\sum_{j=1}^{V} \exp\left(\frac{z_{t,j}}{T}\right)}. \tag{1.12}$$

The temperature controls the entropy and randomness of the emitted sequence:

- **Low Temperature ($T \to 0$ / Greedy Decoding)**: Exaggerates differences between logits. The largest logit absorbs almost all probability mass ($p_{\max} \to 1$). The model makes confident, conservative predictions, but frequently falls into pathological mode collapse—getting trapped in infinite repetitive loops (e.g., `"the the the the"` or repeating identical sentence fragments).
- **Balanced Temperature ($T \in [0.6, 0.8]$)**: Preserves the syntactic and morphological structure discovered by the network while introducing sufficient stochastic diversity to produce coherent, rich generations. In empirical experiments demonstrated in [**?**], character-level LSTMs trained at $T = 0.7$ generated structurally sound Shakespearean plays (including character names, stage directions, and poetic meter), syntactically compilable C code from the Linux kernel, and valid algebraic geometry LaTeX documents.
- **High Temperature ($T > 1.2$)**: Flattens the probability distribution toward a uniform distribution ($p_c \to 1/V$). The network makes chaotic, exploratory predictions, generating misspelled words, hallucinated punctuation, and unpronounceable character noise.

By inspecting the internal cell activations $\mathbf{h}_t$ of a trained LSTM as it reads character sequences, the empirical investigation in [**?**] revealed that individual hidden units act as interpretable linguistic finite-state machines:

1. **The Quote Detection Neuron**: A single scalar unit in $\mathbf{h}_t$ that fires strongly positive (+1) the moment an opening quotation mark (`"`) is parsed, remains persistently active throughout the enclosed dialogue, and resets immediately to −1 when the matching closing quote mark is encountered.
2. **The Code Indentation / Scope Depth Neuron**: In code generation models, a specific neuron's activation ramps up linearly with every tab or whitespace indentation inside nested loops or conditionals, tracking variable scope depth and resetting when closing brackets (`}`) appear.
3. **The Line Length Tracker**: A unit whose activation monotonically increases across 80 characters, anticipating when to emit the carriage return (`\n`) character to conform to standard formatting conventions.

In practical character sequence modeling, two essential takeaways emerge from the Sherlock Holmes casing demonstration:

- **Causal Lookahead Blindness**: Why did the one-to-one RNN fail to capitalize `"mr."` into `"Mr."` even after 10 epochs? The model operates strictly causally from left to right. When processing the character `'m'`, the model only sees past history. It cannot see into the future to know whether `'m'` begins the proper title `"Mr."` or common words like `"me"`, `"my"`, or `"much"`. Without lookahead, capitalizing `'m'` would be an unsupported guess. Conversely, the model successfully capitalizes `"Holmes"` because it has already observed the preceding period and space (`". h"`), triggering capitalization.

- **Overfitting vs. Generalization Dynamics**: Extending training for hundreds of epochs on a finite corpus (such as Doyle's novel) causes the model to memorize specific character n-grams verbatim. While training loss continues to drop toward zero, the network loses its ability to generalize to novel sentence structures, reinforcing the necessity of early stopping and validation monitoring.

1.5 Gated architectures: LSTM and GRU

To solve the vanishing gradient problem and allow neural networks to retain dependencies over hundreds of time steps, researchers introduced **Gated RNNs**:

1.5.1 Long short-term memory (LSTM)

Introduced in [**?**], the **Long Short-Term Memory (LSTM)** network partitions memory into two separate states:

1. **Cell State $\mathbf{c}_t \in \mathbb{R}^{d_h}$**: The *long-term memory* highway, which runs straight down the entire chain with minimal linear interactions.
2. **Hidden State $\mathbf{h}_t \in \mathbb{R}^{d_h}$**: The *short-term memory*, which acts as the visible output emitted at each step.

Definition 1.1: *LSTM gating equations*

At time step t, given input $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, the LSTM evaluates three sigmoid gates and a candidate state:

$$\text{Forget Gate:} \quad \mathbf{f}_t = \sigma\left(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\right) \in (0,1)^{d_h}, \tag{1.13}$$

$$\text{Input Gate:} \quad \mathbf{i}_t = \sigma\left(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\right) \in (0,1)^{d_h}, \tag{1.14}$$

$$\text{Candidate Cell:} \quad \tilde{\mathbf{c}}_t = \tanh\left(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c\right) \in (-1,1)^{d_h}, \tag{1.15}$$

$$\text{Cell State Update:} \quad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}, \tag{1.16}$$

$$\text{Output Gate:} \quad \mathbf{o}_t = \sigma\left(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\right) \in (0,1)^{d_h}, \tag{1.17}$$

$$\text{Hidden State Update:} \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \in (-1,1)^{d_h}, \tag{1.18}$$

where $\odot$ denotes the element-wise Hadamard product, and $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h + d_{\text{in}}}$ denotes vector concatenation.

Semantic roles of the gating units

Each gate squashes pre-activations through a sigmoid function $\sigma \in [0,1]$, functioning as a continuous valve:

- Multiplying by $0 \implies$ drop/forget everything.
- Multiplying by $1 \implies$ preserve/pass everything.
- **Forget Gate ($\mathbf{f}_t$)**: Regulates what proportion of the past cell state $\mathbf{c}_{t-1}$ to discard. (e.g., when a closing quote or period arrives, the forget gate purges past sentence subjects).
- **Input Gate ($\mathbf{i}_t$)**: Regulates what proportion of the new candidate information $\tilde{\mathbf{c}}_t$ to write into the long-term cell state.
- **Output Gate ($\mathbf{o}_t$)**: Regulates which portions of the internal cell state $\mathbf{c}_t$ should be revealed as visible hidden state $\mathbf{h}_t$ for current output predictions.

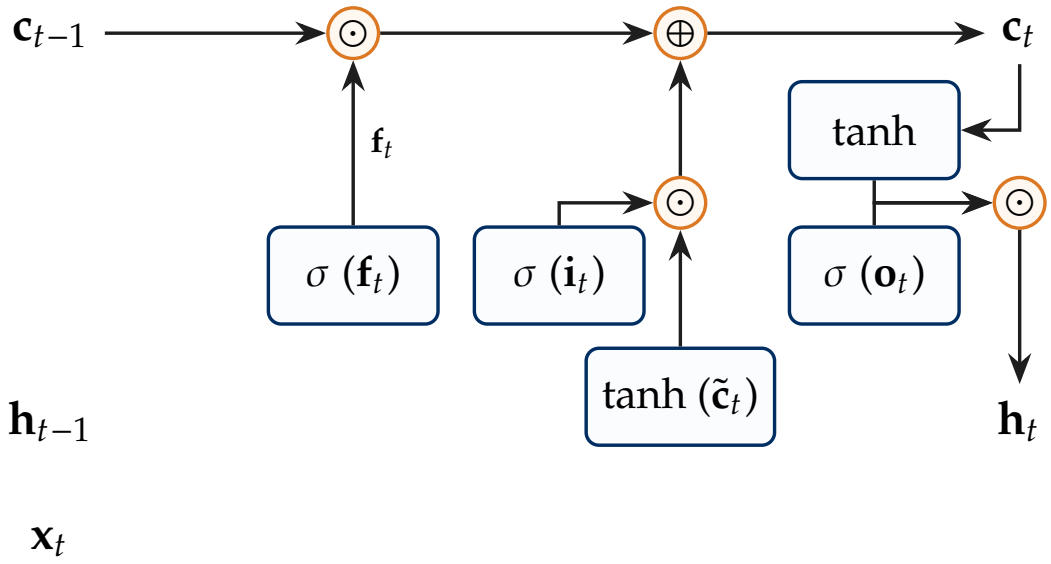


Figure 1.2: The LSTM memory cell architecture. The top horizontal path forms the uninterrupted additive cell state highway $\mathbf{c}_t$ (the Constant Error Carousel). Forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ modulate information flow using multiplicative valves.

Theorem 1.2: *The Constant Error Carousel proof*

In an LSTM, the gradient of the loss with respect to past cell state $\mathbf{c}_{t-1}$ contains an additive direct path:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}}\mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}}\tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}}\mathbf{i}_t. \tag{1.19}$$

When the network sets forget gate $\mathbf{f}_t \approx \mathbf{1}$, the leading derivative collapses to $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$. Consequently, over any arbitrary temporal horizon $T - k$:

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} = \prod_{j=k+1}^{T} \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \prod_{j=k+1}^{T} \mathbf{I} = \mathbf{I}. \tag{1.20}$$

Gradients flow backward across hundreds of time steps without exponential decay. This linear additive gradient highway is the **Constant Error Carousel (CEC)**.

To build physical intuition for the Constant Error Carousel, think of the cell state $\mathbf{c}_t$ as a continuous factory conveyor belt running from the start of the sequence to the end. The standard recurrent transition in a vanilla RNN continually picks up the items, squashes them with weights and tanh, and puts a degraded version back on the belt. Over 50 time steps, the original items are unrecognizable.

In an LSTM, items on the conveyor belt are untouched by default. As the belt passes each station:

1. The forget gate $\mathbf{f}_t$ acts as a discard mechanism: if $\mathbf{f}_t \approx 1$, the past item rides past undisturbed; if $\mathbf{f}_t \approx 0$, it is swept off the belt.
2. The input gate $\mathbf{i}_t$ adds new relevant cargo $\tilde{\mathbf{c}}_t$ onto the belt via simple vector addition ($\oplus$).
3. The output gate $\mathbf{o}_t$ inspects the cargo on the belt, reads off a copy, and displays it to the outside world as hidden state $\mathbf{h}_t$, leaving the underlying cell state intact on the highway.

Because writing is additive rather than multiplicative, information can ride the conveyor belt across hundreds of words without experiencing exponential decay.

1.5.2 Gated recurrent unit (GRU)

Introduced in [**?**], the **Gated Recurrent Unit (GRU)** simplifies the LSTM architecture by merging the cell state and hidden state into a single recurrent state $\mathbf{h}_t$ and coupling the forget and input gates into a single *update gate* $\mathbf{z}_t$:

Definition 1.2: *GRU equations*

At time step t, given input $\mathbf{x}_t$ and prior state $\mathbf{h}_{t-1}$:

$$\text{Update Gate:} \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z\right), \tag{1.21}$$

$$\text{Reset Gate:} \quad \mathbf{r}_t = \sigma\left(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r\right), \tag{1.22}$$

$$\text{Candidate Hidden State:} \quad \tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h\right), \tag{1.23}$$

$$\text{Hidden State Interpolation:} \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{1.24}$$

Physical gate routing and memory modulation

The GRU cell operates through two coupled mechanisms illustrated in Figure **??**:

1. **Reset Gate Modulation ($\mathbf{r}_t$)**: The reset gate vector $\mathbf{r}_t \in (0,1)^{d_h}$ acts as a dynamic filter applied to the previous hidden state $\mathbf{h}_{t-1}$ via Hadamard multiplication ($\mathbf{r}_t \odot \mathbf{h}_{t-1}$). If $\mathbf{r}_t \approx \mathbf{0}$, the candidate state $\tilde{\mathbf{h}}_t$ completely ignores past memory, resetting the representation to depend exclusively on the current token $\mathbf{x}_t$ (e.g., at sentence boundaries or clause resets). If $\mathbf{r}_t \approx \mathbf{1}$, the full history flows into the candidate calculation.

2. **Update Gate Convex Interpolation ($\mathbf{z}_t$)**: Rather than using distinct input and forget gates like the LSTM, the GRU couples memory retention and candidate addition into a single convex combination:
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$
When $\mathbf{z}_t \approx \mathbf{0}$, the unit passes the previous state $\mathbf{h}_{t-1}$ forward unchanged ($1 \odot \mathbf{h}_{t-1} + 0 \odot \tilde{\mathbf{h}}_t = \mathbf{h}_{t-1}$), establishing an unobstructed linear identity shortcut similar to the LSTM Constant Error Carousel. When $\mathbf{z}_t \approx \mathbf{1}$, the unit overwrites the state entirely with the newly proposed candidate $\tilde{\mathbf{h}}_t$.

Parameter count comparison: vanilla RNN vs. LSTM vs. GRU

Comparing the exact parameter counts of recurrent cells for hidden dimension d and input dimension m clarifies the architectural trade-offs across sequence models:

- **Vanilla RNN**: Contains 1 transition set $(\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h) \implies d^2 + dm + d = d(d + m + 1)$.
- **GRU**: Contains 3 gate sets $(\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t) \implies 3 \times d(d + m + 1)$.
- **LSTM**: Contains 4 gate sets $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t, \mathbf{o}_t) \implies 4 \times d(d + m + 1)$.

The GRU reduces parameters by 25% compared to the LSTM while exhibiting comparable empirical performance across diverse sequence modeling benchmarks.

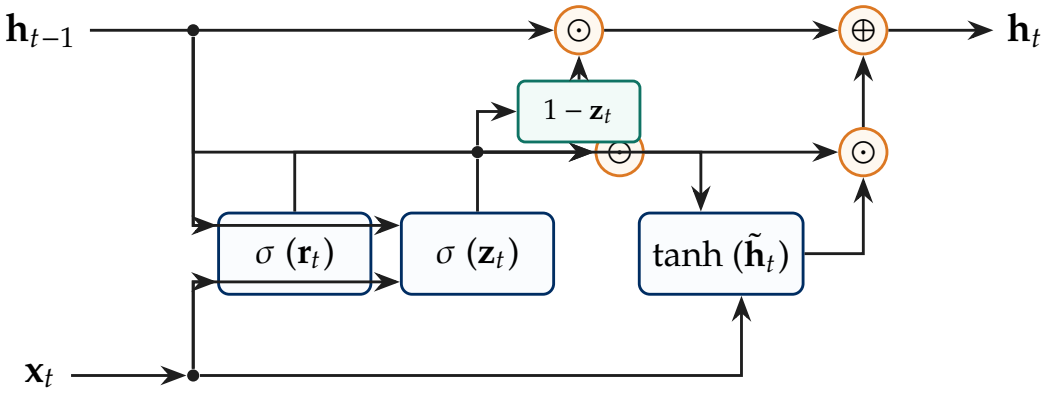


Figure 1.3: The Gated Recurrent Unit (GRU) cell architecture. Reset gate $\mathbf{r}_t$ dynamically filters previous hidden state $\mathbf{h}_{t-1}$ before constructing candidate state $\tilde{\mathbf{h}}_t$. Update gate $\mathbf{z}_t$ controls the convex interpolation $(1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$, unifying long- and short-term memory into a single state vector $\mathbf{h}_t$.

1.6 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines

1.6.1 Limitations of one-to-one sequence mapping

Thus far, we have considered models where each input step produces exactly one output step ($|X| = |Y|$), such as character-level case correction. However, the majority of core NLP tasks (machine translation, question answering, code generation) violate this assumption:

1. **Unequal Sequence Lengths**: In machine translation, source and target languages rarely match in word count:

   English (5 words): *[it] [is] [indeed] [sunny] [today]* $\implies$ Italian (4 words): *[effettivamente] [oggi] [è] [soleggiato]*.

   A one-to-one recurrent model would process 5 tokens and be forced to produce 5 output tokens.

2. **Non-Monotonic Alignment and Causal Blindness**: Natural languages exhibit different word orders. In the example above, the Italian word *"effettivamente"* must be emitted at step 1, but its English counterpart *"indeed"* does not appear until step 3! In a standard one-to-one model, the cell cannot see the future, rendering early word generation impossible.

1.6.2 The encoder-decoder architecture

To overcome these limitations, the **Encoder-Decoder (Seq2Seq)** architecture was introduced in [**?**] and [**?**]:

1. **Encoder**: An LSTM processes the entire variable-length source sequence $(x_1, \ldots, x_N)$ step by step, updating its internal state. The final hidden state $\mathbf{h}_N$ acts as a dense, compressed summary vector encoding the semantics of the whole input sentence.

2. **Decoder**: A separate LSTM initialized with state $\mathbf{h}_N$. The decoder autoregressively emits target tokens step by step. The initial input token is the special Beginning of Sequence symbol ⟨BOS⟩. At each subsequent step t, the input is the token emitted at step $t-1$. Generation continues until the model emits the special End of Sequence symbol ⟨EOS⟩.

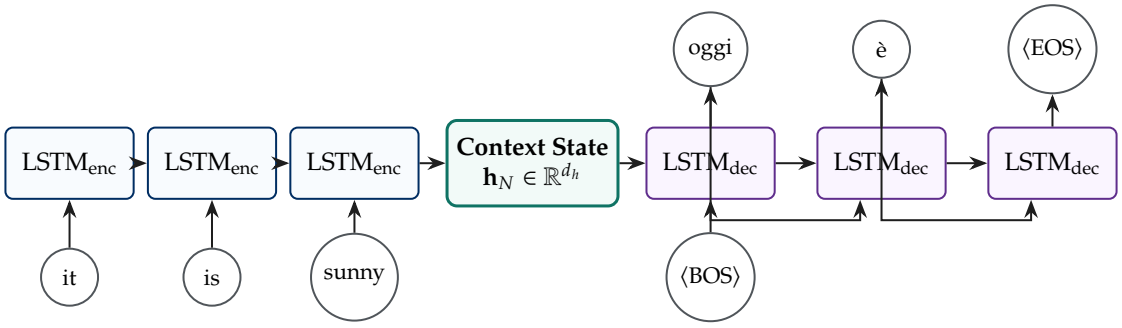


Figure 1.4: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder LSTM compresses the variable-length source sentence into a single context state $\mathbf{h}_N$. The Decoder LSTM is initialized with $\mathbf{h}_N$ and autoregressively emits target tokens from ⟨BOS⟩ until emitting ⟨EOS⟩.

1.6.3 Teacher forcing vs. inference exposure bias

During training, the decoder's predicted token $\hat{y}_{t-1}$ is often incorrect in early epochs. If the model feeds its own faulty prediction into step t, errors compound catastrophically down the sequence.

To stabilize and accelerate training, practitioners use **Teacher Forcing**:

$$\mathbf{x}_{\text{dec},t} = y^*_{t-1} \quad \text{(pass true ground-truth target token instead of predicted } \hat{y}_{t-1}\text{)}. \tag{1.25}$$

Exposure Bias Warning: While Teacher Forcing provides rapid convergence during training, it creates a discrepancy at inference time when ground-truth tokens are unavailable, causing models to be fragile when recovering from early generation mistakes.

1.6.4 Sherlock Holmes case correction revised: one-to-one vs. encoder-decoder

To empirically contrast the representational power of one-to-one recurrent mapping against the Sequence-to-Sequence Encoder-Decoder architecture, we evaluate both models on an identical, challenging character casing sentence:

Input Test Sequence: `"indeed i am sherlock holmes,"`
Target Test Sequence: `"Indeed I am Sherlock Holmes,"`

Theoretical expectations from architectural constraints

This evaluation sentence contains four distinct capitalization targets, each testing a specific causal and temporal capacity:

1. **`"I"` of `"Indeed"` (beginning of quote)**: Should be capitalized because it is immediately preceded by an opening quotation mark (`"`). A one-to-one model can easily learn this local transition, as the trigger character is already present in its past recurrent state $\mathbf{h}_{t-1}$.

2. **`"I"` of `"I"` (standalone subject pronoun)**: Must be capitalized because it functions as the first-person singular pronoun. A one-to-one model is structurally blind here: when reading `'i'`, the network only knows that a word is beginning with `'i'`. It cannot determine whether this character is an isolated pronoun or the initial letter of common words like `"in"`, `"it"`, or `"into"` until the subsequent whitespace arrives at step $t+1$. Without future lookahead, capitalizing `'i'` at step t is mathematically unsupported.

3. **`"S"` of `"Sherlock"` (proper noun first name)**: Must be capitalized because it begins the proper noun *Sherlock*. When the one-to-one cell receives `'s'`, it only observes an arbitrary lowercase word beginning with `'s'`. It cannot anticipate that the subsequent characters will form the name `"herlock"`.

4. **`"H"` of `"Holmes"` (proper noun last name)**: After the model parses the preceding token sequence `"sherlock "`, its recurrent state $\mathbf{h}_{t-1}$ retains the memory of the detective's first name. When an `'h'` arrives, the transition matrix $\mathbf{W}_{hh}$ leverages the learned bigram association $\mathbb{P}(\texttt{Holmes} \mid \texttt{Sherlock})$ to correctly capitalize the last name.

Empirical results across training epochs

Table **??** documents the verbatim loss trajectories and generated outputs across training checkpoints from Epoch 1 to Epoch 1070.

Table 1.3: Empirical comparison of character casing dynamics on the Sherlock Holmes corpus: One-to-One Recurrent Mapping vs. Sequence-to-Sequence Encoder-Decoder across training epochs 1 to 1070.

	One-to-One Mapping		Encoder-Decoder Architecture	
Epoch	**Loss**	**Generated Output**	**Loss**	**Generated Output**
0001	5.3212	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	5.4591	wwxxxxxxxxxxxxxxxxxxxxxxxxxxx
0002	3.7157	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	4.0161	axxxxxxxxxxxxxxxxxxxxxxxxxxxx
0003	2.7964	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	2.8268	xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
0004	2.7981	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx	2.7671	xxxxxxxxxxxxxxxxxx
0030	0.2264	"indeed i am sherlock holmes,"	0.2315	"indeed i am sherlock holmes,"
0050	0.0802	"indeed i am sherlock holmes,"	0.0705	"indeed i am sherlock holmes,"
0160	0.0301	"Indeed i am sherlock Holmes,"	0.0259	"Indeed i am sherlock Holmes,"
0530	0.0224	"Indeed i am sherlock Holmes,"	0.0197	"Indeed I am sherlock Holmes,"
1070	0.0232	"Indeed i am sherlock Holmes,"	0.0156	"Indeed I am Sherlock Holmes,"

Qualitative phase transitions and convergence trade-offs

The comparative trajectories in Table **??** reveal three fundamental pedagogical principles:

1. **The Causal Barrier of One-to-One Recurrence**: The one-to-one model plateaus at Epoch 160 with loss 0.0301 and cannot improve further through Epoch 1070 (loss 0.0232). As predicted, it successfully capitalizes `"Indeed"` (from the quote mark) and `"Holmes"` (from the preceding `"sherlock"`), but remains permanently trapped producing lowercase `"i"` and `"sherlock"`. Because the lockstep recurrent cell must emit output $\hat{y}_t$ simultaneously with reading x_t, it suffers from irreversible causal lookahead blindness.

2. **Sequence-Level Information Integration in Seq2Seq**: The Encoder-Decoder model overcomes causal blindness by decoupling sequence ingestion from sequence emission. The encoder reads the entire string before generation begins. Consequently, the context vector $\mathbf{h}_N$ encodes global contextual features: it "remembers" that `"i"` is bounded by whitespace on both sides (making it the isolated pronoun `"I"`) and that `"s"` begins the full proper name `"sherlock"`. By Epoch 530, the decoder resolves `"Indeed I am sherlock Holmes,"` (loss 0.0197), and by Epoch 1070 it fully resolves all four capitalization targets: `"Indeed I am Sherlock Holmes,"` with superior loss 0.0156.

3. **The Sample Complexity Trade-off (**$100\times$ **Training Epochs)**: Why does the Encoder-Decoder require 1070 epochs when the simple character model converged in 10 epochs? In one-to-one mapping, temporal alignment is strictly hardcoded: character t at the input maps directly to character t at the output. In Seq2Seq, the model is given zero alignment hints; it must simultaneously discover: (i) how to compress the entire sentence into vector $\mathbf{h}_N$, (ii) how to decode variable-length autoregressive tokens, and (iii) how to align output positions with latent source semantics. Learning this unconstrained non-monotonic mapping from scratch drastically increases optimization difficulty and sample complexity.

1.7 Persistent bottlenecks of gated RNNs and the road to transformers

While LSTMs and Encoder-Decoder architectures dominated sequence modeling from 2014 to 2017, three irreducible physical and computational bottlenecks emerged that ultimately necessitated a revolutionary paradigm shift:

In the canonical Seq2Seq framework, an input sequence $(x_1, \dots, x_{T_x})$ of arbitrary length and linguistic entropy is forced to compress into a single continuous vector of fixed dimensionality:

$$\mathbf{h}_{T_x} = \text{Encoder}(x_1, \dots, x_{T_x}) \in \mathbb{R}^{d_h}. \tag{1.26}$$

By Shannon's source coding and rate-distortion theory, a finite-dimensional vector in floating-point precision has bounded information capacity. As sequence length increases beyond $T_x \geq 30$ tokens, the recurrent state suffers from catastrophic information dilution: earlier tokens (often containing essential syntactic subjects and conditioning context) are progressively overwritten by later inputs.

Modern deep learning hardware accelerators (e.g., NVIDIA Hopper/Blackwell GPUs, Google TPUs) derive their performance from massive SIMD (Single Instruction, Multiple Data) parallelism across thousands of tensor cores. Recurrent architectures impose an unbreakable temporal recurrence dependency:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t). \tag{1.27}$$

Computing hidden state $\mathbf{h}_t$ strictly requires the completion of state $\mathbf{h}_{t-1}$. Consequently, an RNN cannot parallelize training across the temporal dimension T. While convolutional and attention layers process the entire sequence in $O(1)$ parallel sequential steps via batched matrix multiplications, RNNs require $O(T)$ sequential operations, rendering pre-training on multi-trillion-token web-scale corpora computationally intractable.

In an unrolled recurrent graph, the shortest path between token x_1 and prediction y_T scales linearly with sequence length: $O(T)$. Every additional step introduces an intermediate Jacobian multiplication $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Even with the additive Constant Error Carousel of LSTMs, error signals become noisy and diffuse over horizons exceeding 100 to 200 tokens.

The historical progression toward self-attention

These structural ceilings precipitated a two-stage intellectual revolution:

1. **Stage 1: Soft Alignment [?]**: Additive Attention shattered the information vector bottleneck by maintaining direct memory access to all encoder annotations $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$. However, because the underlying backbone remained recurrent, training was still bound to the sequential execution wall.

2. **Stage 2: Attention Is All You Need [?]**: The landmark study demonstrated that if attention provides direct $O(1)$ information flow between any pair of tokens regardless of temporal distance, **recurrence is completely redundant**. Discarding recurrent cells entirely in favor of multi-head self-attention unlocked full GPU sequence parallelization and dynamic contextualization, establishing the modern Transformer architecture.

1.7.1 Breaking the vector bottleneck: Bahdanau additive attention

In the standard Seq2Seq architecture (Section **??**), the encoder must squeeze all grammatical, lexical, and structural information of a source sentence of arbitrary length ($T_x = 40, 80$) into the single final state $\mathbf{h}_{T_x} \in \mathbb{R}^{d_h}$. As sentence length increases, empirical translation performance drops precipitously: the fixed-size hidden vector acts as an extreme information bottleneck.

To shatter this bottleneck, **Additive (Soft) Attention** was introduced in [**?**], allowing the decoder to dynamically search and focus across all encoder hidden states at every generation step.

Mathematical formulation with explicit tensor dimensions

Let the source sequence $(x_1, \ldots, x_{T_x})$ be encoded by a Bidirectional RNN, producing a sequence of source annotation vectors:

$$\mathbf{h}_j = \begin{bmatrix} \overrightarrow{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{bmatrix} \in \mathbb{R}^{2d_h}, \quad \forall j \in \{1, 2, \ldots, T_x\}. \tag{1.28}$$

At decoder generation step i, let $\mathbf{s}_{i-1} \in \mathbb{R}^{d_{\text{dec}}}$ denote the previous decoder hidden state.

1. **Alignment Energy Score**: The relevance of source token j to target emission i is evaluated by a small feed-forward network parameterized by $\mathbf{W}_a \in \mathbb{R}^{d_a \times d_{\text{dec}}}$, $\mathbf{U}_a \in \mathbb{R}^{d_a \times 2d_h}$, and $\mathbf{v}_a \in \mathbb{R}^{d_a}$:
$$e_{ij} = \mathbf{v}_a^\top \tanh\left(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j\right) \in \mathbb{R}. \tag{1.29}$$
2. **Softmax Attention Weights**: Exponentiating and normalizing across all T_x source positions yields a valid categorical distribution over input tokens:
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad \text{where } \alpha_{ij} \in (0, 1) \text{ and } \sum_{j=1}^{T_x} \alpha_{ij} = 1. \tag{1.30}$$
The scalar α_{ij} represents the degree to which the decoder should attend to source word x_j when generating target word y_i.
3. **Dynamic Context Vector**: The context vector $\mathbf{c}_i$ is computed as the expected annotation under distribution $\boldsymbol{\alpha}_i$:
$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^{2d_h}. \tag{1.31}$$
Unlike standard Seq2Seq where context $\mathbf{h}_{T_x}$ is static, $\mathbf{c}_i$ is dynamically recomputed at every decoder step i, steering the model's focus to the relevant source phrase.
4. **Decoder State and Emission Update**: The decoder hidden state updates by conditioning on the previous token y_{i-1} and dynamic context $\mathbf{c}_i$:
$$\mathbf{s}_i = \text{RNN}_{\text{dec}}\left(\mathbf{s}_{i-1},\ [\mathbf{y}_{i-1}, \mathbf{c}_i]\right), \tag{1.32}$$
and target token probabilities are emitted via $\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_s[\mathbf{s}_i, \mathbf{y}_{i-1}, \mathbf{c}_i] + \mathbf{b}_s)$.

Figure **??** illustrates the Bahdanau Additive Attention architecture.

The lineage of sequence modeling represents one of the most intellectually cohesive arcs in artificial intelligence:

- In 1997, the milestone paper *"Long Short-Term Memory"* [**?**] solved the vanishing gradient problem via the Constant Error Carousel, proving that additive highways preserve error signals indefinitely.
- In 2014, the Gated Recurrent Unit (GRU) and RNN encoder-decoder architectures were formulated in [**?**] for statistical machine translation.
- Simultaneously in 2014, the landmark study *"Sequence to Sequence Learning with Neural Networks"* [**?**] demonstrated that multi-layer deep LSTMs could map sentences across languages end-to-end. Crucially, it was discovered that reversing the source sentence (*"A B C → C B A"*) dramatically improved performance by shortening the initial temporal gradient distance between source and target tokens.

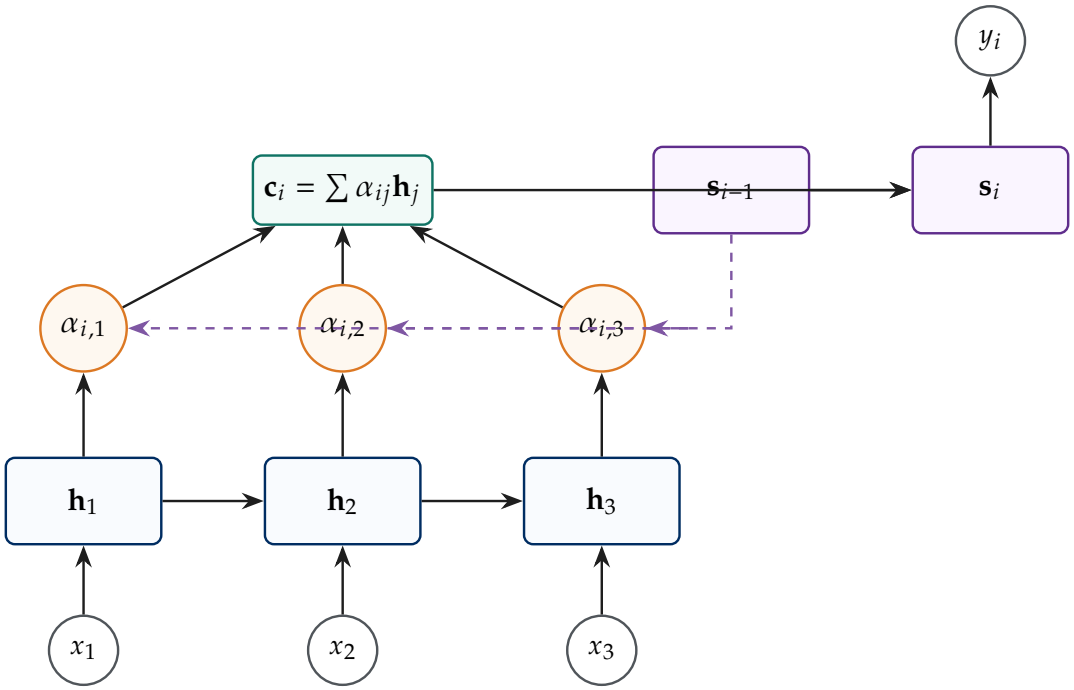


Figure 1.5: The Bahdanau Additive Attention mechanism. At decoder step i, previous decoder state $\mathbf{s}_{i-1}$ is compared against all encoder annotations $\mathbf{h}_j$ via Equation (**??**), producing attention distribution α_{ij}. The weighted sum synthesizes dynamic context vector $\mathbf{c}_i \in \mathbb{R}^{2d_h}$, directly feeding relevant source features into decoder state $\mathbf{s}_i$.

1.7.2 Causal bridge to Chapter 4: the representation problem and continuous word embeddings

Throughout this chapter, we have traced the linear evolutionary arc of sequential neural architectures:

1. **Vanilla RNNs** introduced temporal recurrence and weight sharing to eliminate the fixed-dimensional constraints of feedforward networks, processing elastic sequences via an evolving hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$.

2. **LSTMs and GRUs** resolved exponential vanishing gradients along the temporal chain via the Constant Error Carousel ($\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$) and multiplicative gating highways, enabling gradient propagation across extended sequences.

3. **Seq2Seq Encoder-Decoder Pipelines** overcame the causal lookahead blindness of one-to-one recurrent cells by mapping variable-length input sequences into dynamic variable-length output sequences.

4. **Bahdanau Additive Attention** shattered the single-vector bottleneck ($\mathbf{h}_N \in \mathbb{R}^{d_h}$) by granting the decoder dynamic soft addressing across all encoder annotations ($\mathbf{c}_i = \sum_j \alpha_{ij}\mathbf{h}_j$).

Despite these mathematical triumphs in temporal modeling, every recurrent transition $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$ rests upon a fundamental, unaddressed prerequisite: *the input* $\mathbf{x}_t$ *must be a numeric vector in* $\mathbb{R}^{d_{in}}$. In our pedagogical character casing example, we used a toy ASCII one-hot vector $\mathbf{x}_t \in \{0,1\}^{256}$. However, scaling sequence models to real-world natural language or software engineering source code reveals three catastrophic failure modes of local one-hot representations:

1. **Dimensionality Explosion**: Realistic vocabularies contain $V = |\mathcal{V}| \geq 50{,}000$ to $1{,}000{,}000$ unique words. Representing tokens as one-hot vectors $\mathbf{e}_i \in \{0,1\}^V$ forces the recurrent

input projection matrix $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times V}$ to balloon into millions of parameters, causing severe memory bloat and extreme sample inefficiency.

2. **Orthogonal Metric Collapse**: Standard basis vectors are strictly mutually orthogonal ($\mathbf{e}_i^\top \mathbf{e}_j = 0$ for all $i \neq j$) and uniformly equidistant ($d_E(\mathbf{e}_i, \mathbf{e}_j) = \sqrt{2}$). The recurrent cell receives zero geometric signal that *"dog"* and *"puppy"*, or keywords `"while"` and `"for"`, share syntactic or semantic roles.

3. **Zero Gradient Parameter Sharing**: During backpropagation through time (BPTT), updating weights for one-hot vector $\mathbf{e}_i$ provides absolutely zero learning transfer to semantically similar words $\mathbf{e}_j$, requiring astronomical training data to observe every word in every temporal context.

While recurrent networks solved the *temporal dynamics* problem across sequences, they are crippled without a dense, low-dimensional, continuous metric space $\phi : \mathcal{V} \to \mathbb{R}^d$ ($d \ll V$) where geometric proximity directly encodes syntactic and semantic similarity.

This fundamental representational challenge leads directly to Chapter **??**: *Distributed Word Representations and Word2Vec*.